

Exercise 5 – Reactive Forms

Install Reactive Forms

```
npm install
```

app.config.ts

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideForms, provideReactiveForms } from '@angular/forms';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideForms(),
    provideReactiveForms()
  ]
};
```

Generate Component

```
ng generate component pages/reactive-enrollment
```

app.routes.ts

```
import { Routes } from '@angular/router';
import { HomeComponent } from './pages/home/home.component';
import { CourseListComponent } from './pages/course-list/course-list.component';
import { StudentProfileComponent } from './pages/student-profile/student-profile.component';
```

```
import { EnrollmentFormComponent } from './pages/enrollment-form/enrollment-form.component';

import { ReactiveEnrollmentComponent } from './pages/reactive-enrollment/reactive-enrollment.component';


export const routes: Routes = [
  {path:'',component:HomeComponent},
  {path:'courses',component:CourseListComponent},
  {path:'profile',component:StudentProfileComponent},
  {path:'enroll',component:EnrollmentFormComponent},
  {path:'reactive',component:ReactiveEnrollmentComponent}
];
```

reactive-enrollment.component.ts

```
import { Component } from '@angular/core';

import { FormArray, FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';


@Component({
  selector:'app-reactive-enrollment',
  standalone:true,
  imports:[ReactiveFormsModule],
  templateUrl:'./reactive-enrollment.component.html',
  styleUrls:['./reactive-enrollment.component.css']
})
export class ReactiveEnrollmentComponent{

  constructor(private fb:FormBuilder){}
```

```
enrollmentForm=this.fb.group({
  name:['',Validators.required],
  email:['',[Validators.required,Validators.email]],
  mobile:['',[Validators.required,Validators.pattern('[0-9]{10}')]],
  branch:['',Validators.required],
  semester:['',Validators.required],
  skills:this.fb.array([
    this.fb.control('',Validators.required)
  ])
});
```

```
get skills(){
  return this.enrollmentForm.get('skills') as FormArray;
}
```

```
addSkill(){
  this.skills.push(this.fb.control('',Validators.required));
}
```

```
removeSkill(index:number){
  this.skills.removeAt(index);
}
```

```
submit(){
  console.log(this.enrollmentForm.value);
  this.enrollmentForm.markAllAsTouched();
}
```

```
reset(){
  this.enrollmentForm.reset();
  this.skills.clear();
  this.skills.push(this.fb.control('',Validators.required));
}

}
```

reactive-enrollment.component.html

```
<h2>Reactive Enrollment Form</h2>

<form [formGroup]="enrollmentForm" (ngSubmit)="submit()">

  <label>Name</label>

  <input type="text" formControlName="name">

  <div
    *ngIf="enrollmentForm.get('name')?.invalid&&enrollmentForm.get('name')?.touched">
    Name Required
  </div>

  <label>Email</label>

  <input type="email" formControlName="email">

  <div
    *ngIf="enrollmentForm.get('email')?.invalid&&enrollmentForm.get('email')?.touched">
    Valid Email Required
  </div>
```

```
<label>Mobile</label>
```

```
<input type="text" formControlName="mobile">
```

```
<div
```

```
*ngIf="enrollmentForm.get('mobile')?.invalid&&enrollmentForm.get('mobile')?.touched
">
```

```
Enter 10 Digit Mobile Number
```

```
</div>
```

```
<label>Branch</label>
```

```
<input type="text" formControlName="branch">
```

```
<label>Semester</label>
```

```
<input type="number" formControlName="semester">
```

```
<h3>Skills</h3>
```

```
<div formArrayName="skills">
```

```
<div *ngFor="let skill of skills.controls;let i=index">
```

```
<input [formControlName]="i">
```

```
<button type="button" (click)="removeSkill(i)">Remove</button>
```

```
</div>
```

```
</div>
```

```
<button type="button" (click)="addSkill()">Add Skill</button>
```

```
<br><br>
```

```
<button type="submit" [disabled]="enrollmentForm.invalid">Submit</button>
```

```
<button type="button" (click)="reset()">Reset</button>
```

```
</form>
```

```
<div *ngIf="enrollmentForm.valid">
```

```
<h3>Submitted Data</h3>
```

```
<p>Name : {{enrollmentForm.value.name}}</p>
```

```
<p>Email : {{enrollmentForm.value.email}}</p>
```

```
<p>Mobile : {{enrollmentForm.value.mobile}}</p>
```

```
<p>Branch : {{enrollmentForm.value.branch}}</p>
```

```
<p>Semester : {{enrollmentForm.value.semester}}</p>
```

```
<ul>
```

```
<li *ngFor="let skill of enrollmentForm.value.skills">
```

```
{{skill}}
```

```
</li>
```

```
</ul>
```

```
</div>
```

reactive-enrollment.component.css

```
form{
```

```
width:450px;
```

```
padding:20px;
```

```
border:1px solid #ccc;
```

```
border-radius:10px;
}
label{
display:block;
margin-top:15px;
font-weight:bold;
}
input{
width:100%;
padding:8px;
margin-top:5px;
}
button{
margin-top:10px;
margin-right:10px;
padding:8px 16px;
}
div{
margin-top:5px;
}
```

header.component.html

```
<header>
<h1>{{title}}</h1>
<nav>
<a routerLink="/">Home</a>
<a routerLink="/courses">Courses</a>
<a routerLink="/profile">Profile</a>
```

Template Form

Reactive Form

</nav>

</header>

Run

ng serve

Open:

http://localhost:4200/reactive

Output Screenshot:

The screenshots illustrate the functionality of the Reactive Enrollment Form:

- 1. Reactive Enrollment Form (Initial State):** Shows the form with fields for Name, Email, Mobile, Branch, Semester, and Skills. A green 'Add Skill' button is visible.
- 2. Valid Data Entered with Multiple Skills:** Shows the form with valid data and a list of skills. A green 'Form is Valid' message is displayed.
- 3. Submitted Details Displayed:** Shows the 'Submitted Details' page with a green success message and the user's details.
- 4. Validation Errors (Invalid Data):** Shows the form with red error messages for invalid data.
- 5. Add / Remove Skills Dynamically (FormArray):** Shows the form with a dynamic list of skills that can be added or removed.
- 6. Form Reset to Initial State:** Shows the form after being reset to its initial state.

Key Outcomes Demonstrated

- Reactive Enrollment Form is displayed.
- Validation is handled using Reactive Forms.
- All fields (Name, Email, Mobile, Branch, Semester) are validated.
- Users can dynamically add or remove skills using FormArray.
- Submit button is enabled only when the form is valid.
- Submitted details including skills are displayed.
- Reset button clears the form and restores initial state.
- Complete implementation using Angular Reactive Forms.